From Noise to Diversity: Random Embedding Injection in LLM Reasoning

Anonymous Author(s)
Affiliation
Address
email

Abstract

Recent *soft prompt* research has tried to improve reasoning by inserting trained vectors into LLM inputs, yet whether the gain comes from the *learned content* or from the *act of injection itself* has not been carefully separated. We study Random Soft Prompts (RSPs), which drop the training step entirely and append a freshly drawn random vector to the input. Each RSP vector is sampled from an isotropic Gaussian fitted to the entrywise mean and variance of the pretrained embedding table; it carries no learned content, and yet matches the accuracy of trained soft prompts on math reasoning benchmarks. The mechanism unfolds in two stages: because attention has to absorb a never-seen-before random position, the distribution over the first few generated tokens flattens and reasoning trajectories *branch*, and as generation continues this influence dilutes naturally so the response *converges toward the goal*. We show that during inference RSPs lift early-stage token diversity and, combined with temperature sampling, widen Pass@N — the probability that at least one out of N attempts is correct. Beyond inference, we carry the same effect into DAPO training and demonstrate practical gains. Our contributions are: (i) a new finding that purely training-free random embedding injection alone drives latent reasoning; (ii) a theoretical and empirical validation of the underlying mechanism; and (iii) an extension from inference to training.

1 Introduction

As large language models (LLMs) scale, full fine-tuning becomes prohibitively expensive, motivating parameter-efficient methods that adapt frozen models with few extra parameters [Hu et al., 2022, Houlsby et al., 2019]. Among these, *soft prompts* prepend or insert learnable continuous embeddings into the input, steering behavior through attention [Li and Liang, 2021, Lester et al., 2021]. The paradigm is actively adopted for LLM mathematical reasoning [Kang et al., 2026, Xu et al., 2025, Ye et al., 2026, Hao et al., 2025, Zhang et al., 2026]: despite diverse designs, these methods share a common structure — injecting out-of-vocabulary continuous embeddings and *optimizing* them for downstream performance. Such optimization is task- and dataset-specific, and prior work generally interprets the resulting gains as a consequence of the trained values of these embeddings.

However, we observed that optimization is not necessary for these effects: injecting untrained random embeddings can also lift downstream performance. For instance, applying a chat template to a base model not fine-tuned with such format causes a prompt-format mismatch that degrades reasoning, but appending random embeddings to such mismatched inputs mitigates this degradation. If simple noise were merely shaking the model, accuracy should worsen instead. Therefore, the injection itself, independent of any learned content, alters model behavior in non-trivial ways. Existing studies focus on end-task accuracy and have not separated the effects of injection from those of optimization, leaving how embedding injection reshapes the output distribution insufficiently explored.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

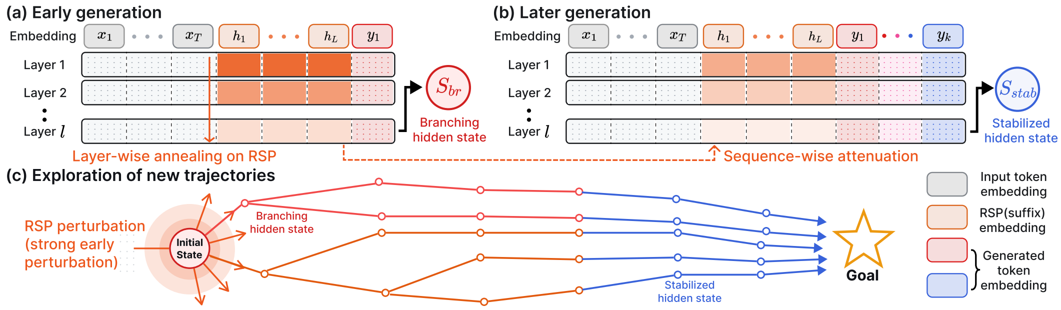


Figure 1: Conceptual overview of RSP-induced trajectory diversity. The hidden states shown are the final-layer outputs that drive the next-token distribution. (a) Early decoding: RSP induces a *branching hidden state* (red) with a diverse output distribution. (b) Later decoding: cache growth attenuates the RSP attention mass, yielding a *stabilized hidden state* (blue). (c) As generation proceeds, the multiple trajectories opened by early RSP perturbation gradually stabilize into goal-directed completions.

We analyze the effect of training-free Random Soft Prompts (RSPs) on LLM reasoning. RSPs are continuous vectors drawn from an isotropic Gaussian fitted to the entrywise mean and variance of the pretrained embedding table, with each rollout receiving an *independent* draw that enables broader exploration. The benefit appears most strongly during early decoding where wider exploration is needed: a single RSP injection induces hidden states that *branch* into diverse trajectories, and as decoding deepens these trajectories *stabilize* into goal-directed completions. Empirically, RSPs lift early-stage entropy and, combined with temperature sampling, expand the Pass@N trajectory diversity, and we further show that the same exploration benefit carries from inference into DAPO [Yu et al., 2025] training. Our contributions are: (i) the new finding that *training-free* random embedding injection alone drives latent reasoning capabilities, suggesting much of the effect attributed to learned soft prompts may come from the *act of injection itself*; (ii) theoretical and empirical validation of the mechanism; and (iii) an extension from inference to training, demonstrating practical applicability.

2 Background

2.1 Soft Prompt Methods for LLM Reasoning

Existing *soft prompt* methods all share a common structure: they inject out-of-vocabulary continuous embeddings and *optimize* them under task-specific objectives. Because evaluation is anchored to end-task accuracy, the contributions of "injection itself" and "optimization" are not separated, and how plain embedding injection reshapes model behavior remains an open question. As parameter-efficient alternatives to full fine-tuning, Prefix-Tuning [Li and Liang, 2021], Prompt Tuning [Lester et al., 2021], and P-tuning [Liu et al., 2024] reached near-fine-tuning performance using learnable continuous vectors, after which LLM-reasoning-specific variants quickly followed. TTSV [Kang et al., 2026] optimizes prefix embeddings at test time via trajectory entropy minimization; SoftCoT [Xu et al., 2025] replaces explicit chain-of-thought tokens with soft tokens generated by an assistant model; LTPO [Ye et al., 2026] optimizes the soft prompt per problem to maximize its own confidence on that problem; COCONUT [Hao et al., 2025] feeds the model's own hidden states back as input embeddings; MemGen [Zhang et al., 2026] uses embedding vectors as experience memory; and Pause tokens [Goyal et al., 2024] insert learnable tokens to provide additional computation steps.

2.2 Noise Injection in Neural Networks

Neural networks have incorporated noise at several stages and locations. During training, dropout [Srivastava et al., 2014] randomly deactivates *hidden activations*, and NEFTune [Jain et al., 2024] *adds* Gaussian noise to *input embeddings* during instruction fine-tuning to improve instruction following. At inference time, randomized smoothing [Cohen et al., 2019] injects Gaussian noise into *raw inputs* to obtain certified robustness, and SmoothLLM [Robey et al., 2025] perturbs prompts at the *discrete character* level to defend against jailbreaking. In RL, NoisyNet [Fortunato et al., 2018] *adds* learnable, trained noise to *network weights* to aid exploration.

RSP *appends* noise to *embeddings*, and differs from these in three respects. **(1)** It operates without any training, purely at inference, and does not modify model parameters (unlike dropout, NEFTune, and NoisyNet). **(2)** It preserves the original input and appends the embedding within a single forward pass, so it does not require the "multiple noisy passes plus aggregation" that robustness methods need. **(3)** It uses vectors drawn directly from the pretrained embedding statistics, with no learned parameters at all (NoisyNet, by contrast, learns its noise parameters). The goal of RSP is not output stability but *exploration*, and this paper analyzes how that difference operates at the attention level.

3 Random Soft Prompts and Why They Work

This section states the mechanism used in the paper. A *rollout* means one full autoregressive decode under one sampled RSP. Within a rollout, an RSP enters the hidden state only through attention; the same scalar attention mass that enables an early influence is constrained by a cache-growth envelope as real tokens accumulate. Across rollouts, independent resampling can accumulate early branch differences into Pass@N gains under the task-side assumptions stated below. We keep the main statements and intuition here; further derivations are in Appendix D.

3.1 Random Soft Prompt

We start with the object being analyzed. Let $\mathbf{W}_E \in \mathbb{R}^{V \times d}$ denote the pretrained token-embedding matrix, where V is the vocabulary size and d is the hidden dimension. Write its entrywise mean and standard deviation as $\mu_E := \frac{1}{Vd}\sum_{i,k}[\mathbf{W}_E]_{i,k}$ and $\sigma_E := \mathrm{std}(\mathbf{W}_E)$. An RSP of length L is a sequence of continuous vectors $\mathbf{H} = [h_1, \ldots, h_L] \in \mathbb{R}^{L \times d}$ drawn independently from

$$h_j \;\sim\; \mathcal{N}(\mu_E \mathbf{1}_d,\; \sigma_E^2 \mathbf{I}_d), \quad j = 1, \ldots, L. \tag{1}$$

The centered form $\bar{h}_j := h_j - \mu_E \mathbf{1}_d \sim \mathcal{N}(0, \sigma_E^2 \mathbf{I}_d)$ is a zero-mean isotropic Gaussian. The RSP can be concatenated with the input embeddings $\mathbf{X}$ at one of three positions: *prefix* ($[\mathbf{H}; \mathbf{X}]$), *infix* ($\mathbf{H}$ inserted within $\mathbf{X}$), or *suffix* ($[\mathbf{X}; \mathbf{H}]$). These positions reflect the injection strategies in prior work: TTSV [Kang et al., 2026] prepends as *prefix*, SoftCoT [Xu et al., 2025] and LTPO [Ye et al., 2026] insert as *infix* between user and assistant turns, and MemGen [Zhang et al., 2026] appends as *suffix*. For repeated-sampling protocols, RSP draws and decoding randomness are independent across trials. The role of this randomness is not to imitate the embedding distribution but to give repeated rollouts *independent* perturbations, which is what exploration requires.

3.2 Exploration: how RSP enters the hidden state

We first ask how a single RSP injection perturbs the hidden state. RSP does not add to the hidden state directly; it enters only through attention weights, so its instantaneous influence within one head can collect into a single quantity: the attention mass assigned to RSP tokens.

Concretely, consider one attention head in self-attention layer ℓ with $n \geq 1$ unmasked real tokens, $L \geq 1$ RSP tokens, finite attention logits, attention weights $\alpha_{ij}^{(\ell)}$, and value vectors $\mathbf{v}_j^{(\ell)}, \mathbf{v}_{r_j}^{(\ell)}$ on the real and RSP sides. Define the RSP attention mass $w_{r,i}^{(\ell)} := \sum_{j=1}^{L} \alpha_{i,n+j}^{(\ell)}$. Since the real-token mass is then positive, the head output decomposes *exactly* as

$$\mathbf{o}_i^{(\ell)} \;=\; (1 - w_{r,i}^{(\ell)})\, \tilde{\mathbf{o}}_i^{(\ell)} \;+\; \boldsymbol{\eta}_i^{(\ell)}, \quad \tilde{\mathbf{o}}_i^{(\ell)} := \sum_{j=1}^{n} \frac{\alpha_{ij}^{(\ell)}}{1 - w_{r,i}^{(\ell)}}\, \mathbf{v}_j^{(\ell)}, \quad \boldsymbol{\eta}_i^{(\ell)} := \sum_{j=1}^{L} \alpha_{i,n+j}^{(\ell)} \mathbf{v}_{r_j}^{(\ell)}. \tag{2}$$

Derivation of Eq. (2). Write the head output as $\sum_{j=1}^{n} \alpha_{ij}^{(\ell)} \mathbf{v}_j^{(\ell)} + \sum_{j=1}^{L} \alpha_{i,n+j}^{(\ell)} \mathbf{v}_{r_j}^{(\ell)}$, then factor $1 - w_{r,i}^{(\ell)} = \sum_{j=1}^{n} \alpha_{ij}^{(\ell)} > 0$ out of the real-token sum. No approximation beyond the softmax-normalization identity is used. □

The same scalar $w_{r,i}^{(\ell)}$ controls two things at once. The attenuation ratio $1 - w_{r,i}^{(\ell)}$ on the real-token signal and the upper bound $\|\boldsymbol{\eta}_i^{(\ell)}\| \leq w_{r,i}^{(\ell)} \, \max_j \|\mathbf{v}_{r_j}^{(\ell)}\|$ on the random contribution are both tied to it. The RSP-induced variation reduces to tracking a single number in $[0, 1]$. Early in decoding the cache is short and $w_{r,i}^{(\ell)}$ can be nontrivial, so the same identity that bounds the random contribution also lets one sampled RSP perturb early next-token decisions.

3.3 Annealing: KV cache growth dilutes RSP attention

The natural next question is how this scalar behaves as decoding proceeds. In autoregressive generation, the KV cache adds one real-token term at each step while the number of RSP terms stays fixed. Under a fixed real-vs-random logit gap, this cache growth gives an upper envelope.

Theorem 1 (Attention-mass decay under cache growth)**.** *For a query attending $n \geq 1$ real tokens and $L \geq 1$ RSP tokens with finite logits, let $\Delta_i^{(\ell)} := \min_{j \in [n]} \mathbf{q}_i^\top \mathbf{k}_j - \max_{j' \in [L]} \mathbf{q}_i^\top \mathbf{k}_{r_{j'}}$ denote the pre-scale logit gap between real and random tokens. In scaled dot-product attention,*

$$w_{r,i}^{(\ell)} \;\leq\; \frac{L}{L + n\,\exp(\Delta_i^{(\ell)}/\sqrt{d})},$$

and the right-hand side is strictly decreasing in n and tends to 0 at fixed L and $\Delta_i^{(\ell)}$.

Proof. Let $s_j := \mathbf{q}_i^\top \mathbf{k}_j/\sqrt{d}$, $s_{r_{j'}} := \mathbf{q}_i^\top \mathbf{k}_{r_{j'}}/\sqrt{d}$, $s_{r,\max} := \max_{j'} s_{r_{j'}}$, and $R := \sum_{j'=1}^{L} \exp(s_{r_{j'}})$. The gap definition gives $s_j \geq \Delta_i^{(\ell)}/\sqrt{d} + s_{r,\max}$ for every real j, and $\exp(s_{r,\max}) \geq R/L$ holds because the max dominates the average; combining the two and summing over the n real keys yields $\sum_{j=1}^{n} \exp(s_j) \geq (n/L)\exp(\Delta_i^{(\ell)}/\sqrt{d})\,R$. Substituting into $w_{r,i}^{(\ell)} = R/(\sum_j \exp(s_j) + R)$ gives

$$w_{r,i}^{(\ell)} \;\leq\; \frac{R}{(n/L)\exp(\Delta_i^{(\ell)}/\sqrt{d})\,R + R} \;=\; \frac{L}{L + n\,\exp(\Delta_i^{(\ell)}/\sqrt{d})}.$$

The derivative of the right-hand side in n is $-L\exp(\Delta_i^{(\ell)}/\sqrt{d})/(L + n\exp(\Delta_i^{(\ell)}/\sqrt{d}))^2 < 0$, so the bound is strictly decreasing and tends to 0 as $n \to \infty$. $\square$

This single inequality carries two messages. The RSP length L is the design parameter we choose; it sits in the numerator and sets the *maximum* influence of the perturbation. The real-token count n grows automatically every step and sits in the denominator, so the *envelope* of the influence shrinks under the same L. What the theorem guarantees is the sequence-direction attenuation of the upper envelope at fixed (layer, query position, logit gap), not a layer-direction monotone decay; queries, keys, and hidden states co-evolve, so $\Delta_i^{(\ell)}$ also moves and the realized $w_{r,i}^{(\ell)}$ need not decrease at every step. The accurate reading is that "the envelope compatible with a fixed gap shrinks as n grows." Because Eq. (2) bounds the random contribution by the same scalar, the perturbation magnitude inherits the same envelope. This gives an implicit explore-then-exploit interpretation of decoding without asserting stepwise monotonic decay. Figure 2 verifies the sequence-direction prediction empirically.

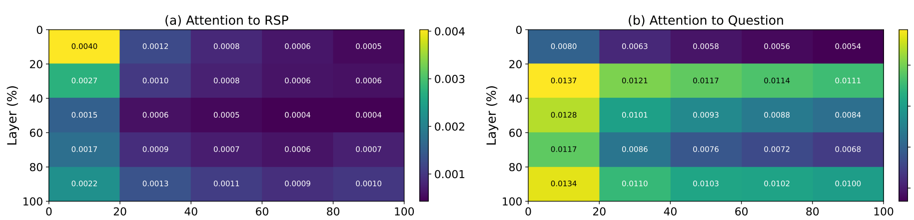


Figure 2: Per-token attention mass on Qwen2.5-Math-7B (suffix, 500 MATH-500 problems, each with an independently sampled RSP). (a) RSP-token attention decreases along the reasoning-position axis, consistent with the cache-growth bound of Theorem 1. The additional decrease along the layer axis is a separate empirical phenomenon suggesting that deeper layers sharpen the real-vs-random logit gap; it does not follow from Theorem 1 alone. (b) Question-token attention remains comparatively uniform. The reported quantity is the per-token mass of Appendix E, equal to $w_{r,i}^{(\ell)}/L$ averaged over heads and samples.

3.4 Performance gain: why independent reruns help

Theorem 1 explains what one rollout can do. The remaining question is performance: why can *fresh* RSP draws across reruns help more than repeating a single fixed perturbation? The answer needs one distributional fact about the prompt law and one separate task-side fact about productive branches.

The first fact is directional. The attention identity above does not require isotropy, but branch opening benefits from a perturbation law that does not under-serve whichever local direction matters for the current prompt. We state this fact for one centered RSP vector; the local surrogate below isolates one such perturbation from the length-L prompt. A minimax statement formalizes the point.

Theorem 2 (Direction-agnostic minimax coverage)**.** *Let D be any zero-mean distribution on $\mathbb{R}^d$ with covariance Σ_D and trace budget $\mathrm{tr}(\Sigma_D) \le \rho^2$. Then $\min_{\|b\|=1} \mathrm{Var}_{h\sim D}(b^\top h) = \lambda_{\min}(\Sigma_D) \le \rho^2/d$, with equality if and only if $\Sigma_D = (\rho^2/d)\mathbf{I}_d$.*

Equality holds for any distribution with isotropic covariance—Gaussianity is not the unique answer. The Gaussian RSP design *borrows the scale from the embedding distribution but discards its directional geometry on purpose*. The isotropic Gaussian additionally has positive density on all of $\mathbb{R}^d$ (full support), so it places no prior weight on any direction (Appendix C); learned fixed prompts ($\Sigma_D = 0$), PCA-aligned noise, vocabulary-token sampling, and embedding-covariance matching are all anisotropic and so under-explore at least one direction.

We now formalize branch opening on the surrogate. Linearizing the logits around $\bar{h} = 0$, define the affine surrogate $z_i^{\mathrm{lin}}(\bar{h}) := z_i + a_i^\top \bar{h}$ with $a_i := \nabla_{\bar{h}} z_i(0)$. For a baseline-excluded token i, the top-K entry region is $\mathcal{G}_{i,K} := \big\{\bar{h} \in \mathbb{R}^d : \#\{j \neq i : z_j^{\mathrm{lin}}(\bar{h}) > z_i^{\mathrm{lin}}(\bar{h})\} \le K-1\big\}$, matching the actual top-K definition.

Proposition 1 (Conditional top-K entry)**.** *If $\mathcal{G}_{i,K}$ contains a nonempty open set, the isotropic Gaussian's full support assigns it positive probability, so $\mu_i := \Pr_{\bar{h}}(\bar{h} \in \mathcal{G}_{i,K}) > 0$.*

This proposition gives a conditional reachability statement: once such a local top-K region exists in the surrogate, the RSP law assigns it positive probability. The existence of that region remains a task-side condition, not a consequence of the mechanism alone. To lift a single-draw opening probability to task accuracy we add two task-side assumptions: (M1) each independent RSP run reaches a correct trajectory with marginal probability at least $p_{\min}(x) > 0$, and (M2) the N runs are mutually independent.

Proposition 2 (Multi-path Pass@N)**.** *Under (M1)–(M2), with N independent RSP runs,* $\Pr\big(\mathrm{Pass@}N \textit{ on } x\big) \;\ge\; 1-(1-p_{\min}(x))^N \;\ge\; 1-\exp(-N\,p_{\min}(x))$.

This division of labor is the main interpretive point. The mechanism can *admit* baseline-excluded candidates into the local candidate set when the corresponding entry region exists; whether an admitted branch is *productive* is task-dependent, which is why $p_{\min}(x) > 0$ is a separate assumption. Sharing one RSP across all samples removes this accumulation, so *independence* is essential. Section 4 tests exactly this signature.

4 Empirical Validation

We examine how the *early hidden-state branching followed by cache-growth stabilization* pattern of Figure 1 surfaces in LLM generation through three scenes: whether RSPs preserve baseline accuracy and even recover it in some settings (§4.2); whether the early diversification and later stabilization suggested by the attention decomposition and the cache-growth envelope are observed in practice (§4.3); and whether Pass@N depends on *independence* of the RSP draw across samples (§4.4).

4.1 Experimental Setup

We evaluate on three mathematical reasoning benchmarks, MATH-500 [Lightman et al., 2024], GSM8K [Cobbe et al., 2021], and AIME24, using LLaMA-3.1-8B-Instruct [Grattafiori et al., 2024] and the Qwen2.5-Math model family [Yang et al., 2024] across instruct models, base models, and base models with mismatched chat templates—seven configurations in total. All experiments use greedy decoding to isolate the effect of RSPs from sampling stochasticity, and the answer-extraction pipeline uses fallbacks to score only the final answer. RSPs follow the definition of §3.1 and are

Table 1: Accuracy (%) across model configurations and benchmarks. Values in parentheses indicate the difference from the baseline. Full per-position breakdown (including infix) is in Appendix A.

	MATH-500			GSM8K			AIME24		
Model	Baseline	RSP(*prefix*)	RSP(*suffix*)	Baseline	RSP(*prefix*)	RSP(*suffix*)	Baseline	RSP(*prefix*)	RSP(*suffix*)
Instruct Models									
LLaMA-3.1-8B-Inst	52.20	45.00 (−7.2)	49.40 (−2.8)	85.75	76.35 (−9.4)	86.73 (+1.0)	6.67	3.33 (−3.3)	10.00 (+3.3)
Qwen2.5-Math-7B-Inst	83.20	81.40 (−1.8)	83.40 (+0.2)	95.45	94.92 (−0.5)	95.68 (+0.2)	13.33	13.33 (+0.0)	16.67 (+3.3)
Qwen2.5-Math-1.5B-Inst	73.00	74.80 (+1.8)	74.20 (+1.2)	85.29	85.29 (+0.0)	84.69 (−0.6)	10.00	13.33 (+3.3)	20.00 (+10.0)
Base Models (Raw Text)									
Qwen2.5-Math-7B	72.20	71.60 (−0.6)	55.60 (−16.6)	84.15	84.31 (+0.2)	54.13 (−30.0)	6.67	16.67 (+10.0)	13.33 (+6.7)
Qwen2.5-Math-1.5B	61.40	64.40 (+3.0)	54.60 (−6.8)	77.86	74.30 (−3.6)	58.61 (−19.3)	16.67	10.00 (−6.7)	3.33 (−13.3)
Base Models + ChatML (Format Mismatch)									
Qwen2.5-Math-7B	52.20	59.20 (+7.0)	70.40 (+18.2)	58.30	56.41 (−1.9)	75.97 (+17.7)	23.33	3.33 (−20.0)	23.33 (+0.0)
Qwen2.5-Math-1.5B	34.40	63.40 (+29.0)	51.00 (+16.6)	37.45	67.02 (+29.6)	50.64 (+13.2)	13.33	20.00 (+6.7)	13.33 (+0.0)

Table 2: Comparison with existing soft prompt methods (TTSV, SoftCoT, LTPO) under unified evaluation. Baseline and RSP values are from Table 1. **Bold** denotes the best result and $\underline{\text{underline}}$ denotes the second best for each model–benchmark pair.

Model	**Benchmark**	**Baseline**	**RSP**	**TTSV**	**SoftCoT**	**LTPO**
Qwen2.5-Math-7B-Instruct	MATH-500	83.20	**84.20**	$\underline{83.80}$	82.80	83.00
	GSM8K	$\underline{95.45}$	**95.68**	95.30	95.00	94.84
	AIME24	13.33	$\underline{16.67}$	**23.30**	$\underline{16.67}$	13.33
Qwen2.5-Math-1.5B-Instruct	MATH-500	73.00	$\underline{75.20}$	$\underline{75.20}$	**76.00**	72.40
	GSM8K	85.29	**85.75**	$\underline{85.70}$	84.46	84.53
	AIME24	$\underline{10.00}$	**20.00**	6.70	6.67	$\underline{10.00}$

concatenated at one of three positions (prefix, infix, suffix) with a freshly drawn $\mathbf{H}$ for each batch. Full experimental details and per-position results are in Appendix A.

4.2 How Does Untrained RSP Compare to Baselines & Optimized Soft Prompts?

We first examine how injecting random embeddings affects reasoning performance. Table 1 reports the accuracy for each benchmark, using the best of Prefix and Suffix injection at $L \in \{10, 15, 20\}$ (the full per-position breakdown including infix is in Appendix A).

On instruct models RSP stays within a few percentage points of the baseline. The most striking pattern is the format-mismatch case: when a ChatML template is applied to a base model not trained with such a format, suffix injection on Qwen2.5-Math-7B recovers $+18.2$ pp on MATH-500 and $+17.7$ pp on GSM8K, and prefix injection on Qwen2.5-Math-1.5B recovers up to $+29.0$ and $+29.6$ pp respectively. If simple noise were responsible, accuracy should have dropped further; the effect goes beyond mere perturbation. We emphasize not that any single position is superior, but that *training-free* RSPs reach a level comparable to trained soft prompts — suggesting much of the effect previously attributed to trained methods may in fact come from the *act of injection itself*. As a working hypothesis, base models appear to fall into abnormally confident wrong attractors on unfamiliar chat tokens, and RSP's early-stage flattening helps the model escape (consistent with the entropy-rise signature in §4.3); quantitative and multi-seed analysis are left for future work.

We further compare RSP with TTSV, SoftCoT, and LTPO — prior soft prompt methods that optimize embeddings under different objectives. Table 2 reports a unified evaluation under the same fallback-based answer-extraction pipeline (Appendix A.2). Despite each method's distinct prompt format and grading scheme, this format-agnostic comparison places RSP on par with the optimized methods without any training.

4.3 How Does RSP Affect the Output Distribution?

We next examine how RSP injection reshapes the output distribution during generation. The setting is Qwen2.5-Math-1.5B-Instruct on MATH-500, and we measure per-token entropy, top-1 probability, and varentropy across generation steps. Figure 3 compares these metrics for the first 5% of generation steps across RSP variants and prior soft prompt methods (LTPO, SoftCoT, TTSV); full curves are in Appendix A.4.

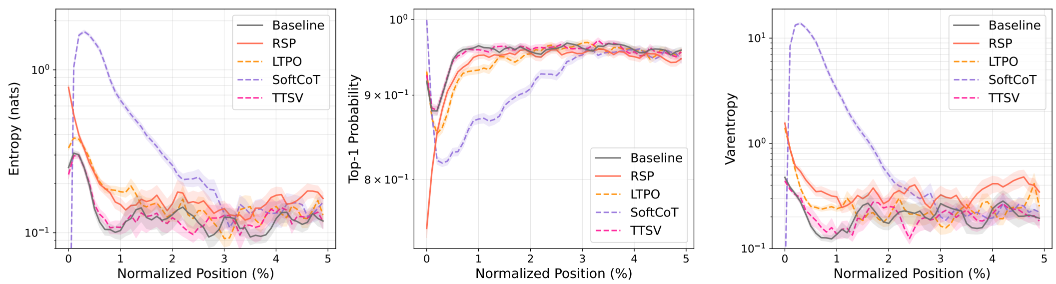


Figure 3: Mean entropy, top-1 probability, and varentropy during the first 5% of generation steps (Qwen2.5-Math-1.5B-Instruct, MATH-500). Shading indicates ± 1 SEM. Solid lines: RSP variants and the baseline. Dashed lines: prior soft prompt methods (LTPO, SoftCoT, TTSV).

Latent prompt methods including RSP share the same signature: *a rise in early-generation entropy followed by convergence to the baseline later in generation*. A natural mechanistic explanation is the out-of-vocabulary (OOV) effect — when continuous embeddings outside the vocabulary enter the input, the model has to attend to a never-before-seen key position on top of its familiar token distribution, flattening the early next-token distribution. All three RSP injection positions show early-stage entropy↑, top-1 probability↓, and varentropy↑, indicating multimodal distributions [Ahmed et al., 2026]. The change is confined to the early stage and all three metrics return to baseline in the middle and final portions, consistent with the *early influence with automatic cache-growth decay* pattern predicted in §3.3.

Existing methods trained under different objectives (LTPO, SoftCoT, TTSV) share this signature. Even TTSV, whose reward explicitly *lowers* mean reasoning-trajectory entropy, shows early-stage entropy comparable to or higher than the baseline. The entropy value itself is not a quality score — higher entropy is not always better; rather, the signature reveals a *shared mechanism* in which OOV embedding injection produces the same fingerprint independently of the optimization objective, supporting the hypothesis that the key driver of latent reasoning is the *act of injection*, not the learned content.

4.4 Does RSP Induce Trajectory Diversity?

Section 4.3 empirically established that RSP reshapes the early-stage output distribution. We now examine whether this shift translates into reasoning diversity at three complementary levels of analysis: token rankings, semantic content of trajectories, and task-level outcomes. The three analyses converge on a directional picture: RSP's first-token perturbation propagates into semantically more diverse reasoning trajectories, which in turn yields greater output diversity at the task level. All three analyses share the setting: Qwen2.5-Math-1.5B-Instruct, MATH-500, suffix injection, and $L = 10$.

Table 3: First-token distribution metrics measured against the baseline at τ=1.0, comparing baselines at higher temperatures (τ=2.0, 3.0) with RSP at τ=1.0. RSP metrics are averaged over 16 random seeds; Acc reports mean $\pm$ std across 16 samples per problem.

Metric	τ=2.0	τ=3.0	**RSP**
Spearman ρ	1.000	1.000	**0.709**
Mass outside top-10	5.18%	29.11%	**21.86%**
JS divergence	0.060	0.218	**0.131**
Acc (%)	20.77 $\pm$ 1.47	1.42 $\pm$ 0.35	**73.30 $\pm$ 1.07**

Token-level: distribution beyond temperature. We quantify how much RSP shifts the first-token distribution — the critical forking point for reasoning trajectories [Wang et al., 2025]. Using the baseline at τ=1.0 as reference, we compare baselines at τ=2.0, 3.0 (16 samples per problem) against RSP at τ=1.0 (averaged over 16 seeds) on four metrics: Spearman rank correlation ρ, probability mass outside the baseline's top-10, Jensen–Shannon divergence, and Pass@1 (definitions in Appendix B). Table 3 shows the contrast: RSP's distributional shift falls between τ=2.0 and τ=3.0 in magnitude, but the mechanism differs — temperature is a monotone transform that preserves ranking ($\rho = 1$ at any τ), while RSP partially preserves but reranks ($\rho = 0.709$). The Pass@1 row exposes the cost: matching RSP's magnitude with temperature toward τ=3.0 collapses accuracy to 1.42%, whereas RSP preserves 73.30%. RSP produces a fundamentally different perturbation than temperature scaling.

Trajectory-level: semantic diversity. Does first-token reranking translate into semantically diverse trajectories? We sample 64 problems from MATH-500 and generate 300 independent trajectories per problem under Baseline and RSP at $\tau = 1.0$. Each trajectory is encoded by BGE-M3 [Chen et al., 2024] into a dense semantic vector, and we compute three metrics: *pairwise cosine distance* (overall semantic spread), *inter-cluster distance* between DBSCAN [Ester et al., 1996] centroids (separation between groups), and *intra-cluster distance* (coherence within groups). Table 4 reveals three concurrent patterns: pairwise distance rises by $1.4\times$, inter-cluster distance jumps by $5.4\times$, and intra-cluster distance is essentially unchanged — the trajectory set becomes more diverse and splits into more separated groups while preserving within-group coherence. RSP also yields larger pairwise distance than baseline on 56 of 64 problems (87.5%).

Table 4: Semantic diversity metrics (64 problems, 300 trajectories per condition).

Metric	Baseline	RSP
Pairwise cos. dist.	0.0612	**0.0879**
Inter-cluster dist.	0.0183	**0.0982**
Intra-cluster dist.	0.0526	0.0528

Outcome-level: Pass@N scaling. Does this token- and trajectory-level diversity translate into task-level outcome diversity, measured by Pass@N [Chen et al., 2021] (the probability that at least one of N sampled solutions is correct)? We compare four conditions on MATH-500 and AIME24 with $N = 16$ samples per problem: *(i)* baseline at $\tau = 1.0$; *(ii)* a single RSP shared across all samples at $\tau = 1.0$; *(iii)* a fresh RSP per sample with greedy decoding; and *(iv)* a fresh RSP per sample at $\tau = 1.0$.

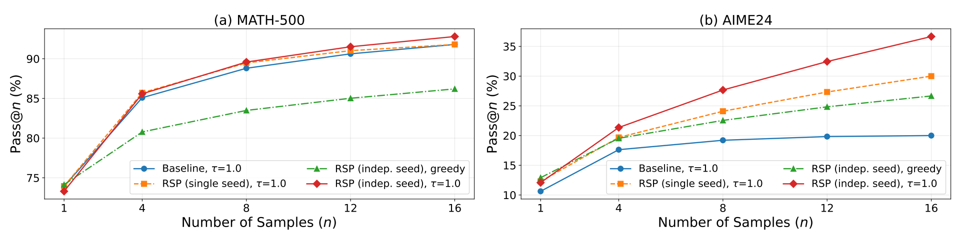


Figure 4: Pass@N scaling on (a) MATH-500 and (b) AIME24 with Qwen2.5-Math-1.5B-Instruct, $N = 16$ samples per problem. Baseline: temperature sampling only. Fixed seed: single RSP shared across samples combined with temperature. Indep. seed: a different RSP per sample, with or without temperature.

Condition *(iv)* achieves the highest Pass@N on both benchmarks, reaching 92.80% on MATH-500 and 36.67% on AIME24 at $N = 16$, while *(ii)* shows no improvement over the baseline — the diversity gain depends on varying the embeddings across samples rather than fixing a single perturbation. Pass@1 stays comparable to the baseline on MATH-500, and on the harder AIME24 *(iv)* even surpasses it, suggesting that independent RSPs combined with temperature sampling expand trajectory diversity without sacrificing single-sample accuracy.

5 Application: DAPO Training with RSP

Sections 3–4 established what RSP does (attention-mediated branch opening) and how it behaves at inference (Pass@N gains, rank-changing diversity unattainable by temperature). Can we use the same effect at training time? Rollout diversity drives learning in sampling-based RL, and DAPO [Yu et al., 2025] is a recent GRPO [Shao et al., 2024] variant explicitly designed to preserve diversity at the loss level. We use it as a stringent testbed: does input-side branch opening compose with loss-side preservation to yield further gains? Details and per-step results are in Appendix F.

DAPO preserves rollout diversity at the loss level by reshaping how rollouts are weighted and clipped during long chain-of-thought RL; RSP induces diversity *before* the loss is computed by perturbing the rollout distribution at the input level (§4.4). With o_i the i-th rollout, $r_{i,t}(\theta) = \pi_\theta(o_{i,t} \mid q, [\mathbf{H}_g,]o_{i,<t})/\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, [\mathbf{H}_g,]o_{i,<t})$ the importance ratio, $\hat{A}_{i,t}$ the group-relative advantage,

$(\varepsilon_{\text{low}}, \varepsilon_{\text{high}})$ the asymmetric clip range, and $\mathbf{H}_g$ the RSP appended to the prompt, the DAPO + RSP objective is

$$\mathcal{J}_{\text{DAPO+RSP}}(\theta) \;=\; \mathbb{E}\left[\frac{1}{\sum_i |o_i|}\sum_{i,t}\min\Big(r_{i,t}\hat{A}_{i,t},\ \text{clip}\big(r_{i,t},\ 1-\varepsilon_{\text{low}},\ 1+\varepsilon_{\text{high}}\big)\hat{A}_{i,t}\Big)\right]. \tag{3}$$

Table 5: Per-benchmark accuracy (%) at each method's peak step on Qwen2.5-Math-7B. **Bold** denotes the better of the two RL methods on each benchmark. Avg is the unweighted five-benchmark mean.

Method	GSM8K	MATH-500	College	Minerva	AIME 24	Avg
Base	57.2	52.6	18.3	13.6	8.6	30.06
DAPO	86.3	78.2	41.4	37.5	22.6	53.20
DAPO + RSP	**87.7**	**78.6**	**42.2**	**39.7**	**23.6**	**54.36**

We train Qwen2.5-Math-7B on a MATH Level-3–5 subset (~8.5K prompts) with SimpleRL-Zoo [Zeng et al., 2025] under Eq. (3): each step uses $G = 8$ rollouts per prompt at τ=1.0, batch 1,024, 4 PPO mini-batches of 256, for 100 steps. *DAPO* is compared against *DAPO + RSP*, where suffix RSP (L=20) is applied during rollouts only and evaluation uses no RSP. We evaluate on five math benchmarks (GSM8K [Cobbe et al., 2021], MATH-500 [Lightman et al., 2024], College Math, Minerva Math [Lewkowycz et al., 2022], AIME 2024) and report the unweighted mean; AIME 2024 uses Avg@32 at τ=1.0, the others greedy.

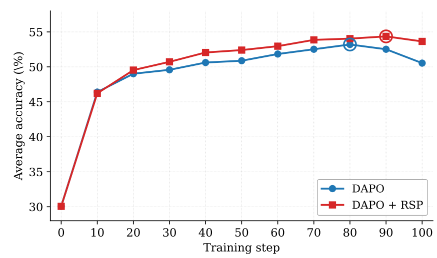


Figure 5: Five-benchmark average accuracy on Qwen2.5-Math-7B. DAPO + RSP reaches a higher peak (step 90) and stays stable through step 100.

Figure 5 and Table 5 show two patterns: a late-training advantage and a delayed early reward growth. DAPO + RSP's average reaches 54.36% at step 90 (vs DAPO's 53.20% peak, +1.16 pp), widening to 53.64% vs 50.54% at step 100 (+3.10 pp), and DAPO's post-peak decline is delayed under RSP; earlier the curves cross around step 20. Both patterns follow from the methods acting at separate stages: DAPO preserves diversity at the loss level over already-generated rollouts, whereas RSP perturbs the hidden states that produce them (§3.2). Input-side perturbation steers the policy away from default trajectories, exposing it to broader learning signals; the cumulative effect is the late-stage ceiling improvement and delayed collapse — the exploration–exploitation dynamics of RL [Sutton and Barto, 2018] induced via hidden-state-level perturbation from the input.

6 Conclusion

We studied Random Soft Prompts (RSPs), isotropic Gaussian vectors fitted to pretrained embedding statistics and injected without training to supply independent perturbations across rollouts. Theoretically, the RSP contribution per attention layer reduces to a single scalar mass with a cache-growth-shrinking envelope, and isotropic resampling reaches top-K entry regions that rank-preserving temperature cannot (§3). Empirically, latent prompt methods share the same entropy signature and the gain vanishes when one RSP is shared across samples — *independence* is the key (§4). The core latent-reasoning pattern thus emerges from purely random input-side perturbation with no learned content, suggesting that much of what learned soft prompts have been credited with may be a by-product of the *act of injection itself*. Future work includes training-time comparisons beyond DAPO, broader domains, and principled RSP-length/position choices.

References

Farhan Ahmed, Yuya Jeremy Ong, and Chad DeLuca. LogitScope: A Framework for Analyzing LLM Uncertainty through Information Metrics. In *ICLR Workshop*, 2026.

Nora Belrose, Igor Ostrovsky, Lev McKinney, Zach Furman, Logan Smith, Danny Halawi, Stella Biderman, and Jacob Steinhardt. Eliciting Latent Predictions from Transformers with the Tuned Lens. *arXiv:2303.08112*, 2023.

Jianlyu Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. In *Findings of ACL*, 2024.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating Large Language Models Trained on Code. *arXiv:2107.03374*, 2021.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training Verifiers to Solve Math Word Problems. *arXiv:2110.14168*, 2021.

Jeremy M. Cohen, Elan Rosenfeld, and J. Zico Kolter. Certified Adversarial Robustness via Randomized Smoothing. In *ICML*, 2019.

Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise. In *KDD*, 1996.

Meire Fortunato, Mohammad Gheshlaghi Azar, Bilal Piot, Jacob Menick, Ian Osband, Alex Graves, Vlad Mnih, Remi Munos, Demis Hassabis, Olivier Pietquin, Charles Blundell, and Shane Legg. Noisy Networks for Exploration. In *ICLR*, 2018.

Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer Feed-Forward Layers Are Key-Value Memories. In *EMNLP*, 2021.

Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. Think before You Speak: Training Language Models with Pause Tokens. In *ICLR*, 2024.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, Danny Wyatt, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Francisco Guzmán, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Govind Thattai, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jack Zhang, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelmer van der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Karthik Prasad, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Kushal Lakhotia, Lauren Rantala-Yeary, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Maria Tsimpoukelli, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kambadur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Ning Zhang, Olivier Duchenne, Onur Çelebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajjwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohan Maheswari, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ronnie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar

Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia Kim, Sergey Edunov, Shaoliang Nie, Sharan Narang, Sharath Raparthy, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stephane Collot, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkez, Vincent Gonguet, Virginie Do, Vish Vogeti, Vítor Albiero, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenyin Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiaofang Wang, Xiaoqing Ellen Tan, Xide Xia, Xinfeng Xie, Xuchao Jia, Xuewei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpierre Coudert, Zheng Yan, Zhengxing Chen, Zoe Papakipos, Aaditya Singh, Aayushi Srivastava, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Amos Teo, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Dong, Annie Franco, Anuj Goyal, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardi, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Ce Liu, Changhan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Cynthia Gao, Damon Civin, Dana Beaty, Daniel Kreymer, Daniel Li, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkang Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Eric-Tuan Le, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Filippos Kokkinos, Firat Ozgenel, Francesco Caggioni, Frank Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Grant Herman, Grigory Sizov, Guangyi Zhang, Guna Lakshminarayanan, Hakan Inan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Hongyuan Zhan, Ibrahim Damlaj, Igor Molybog, Igor Tufanov, Ilias Leontiadis, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Janice Lam, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhie, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou U, Karan Saxena, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelena, Keqian Li, Kiran Jagadeesh, Kun Huang, Kunal Chawla, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Khabsa, Manav Avalani, Manish Bhatt, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Miao Liu, Michael L. Seltzer, Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikhil Mehta, Nikolay Pavlovich Laptev, Ning Dong, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanchandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Rangaprabhu Parthasarathy, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Wang, Russ Howes, Ruty Rinott, Sachin Mehta, Sachin Siby, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Mahajan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy Zha, Shishir Patil, Shiva Shankar, Shuqiang Zhang, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Summer Deng, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Koehler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihailescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiaocheng Tang, Xiaojian

Wu, Xiaolan Wang, Xilun Wu, Xinbo Gao, Yaniv Kleinman, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu Wang, Yu Zhao, Yuchen Hao, Yundi Qian, Yunlu Li, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, Zhiwei Zhao, and Zhiyu Ma. The Llama 3 Herd of Models. *arXiv:2407.21783*, 2024.

Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training Large Language Models to Reason in a Continuous Latent Space. In *ICLR*, 2025.

Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Leng Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yankai Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. OlympiadBench: A Challenging Benchmark for Promoting AGI with Olympiad-Level Bilingual Multimodal Scientific Problems. In *Findings of ACL*, 2024.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-Efficient Transfer Learning for NLP. In *ICML*, 2019.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. In *ICLR*, 2022.

Neel Jain, Ping-yeh Chiang, Yuxin Wen, John Kirchenbauer, Hong-Min Chu, Gowthami Somepalli, Brian R. Bartoldson, Bhavya Kailkhura, Avi Schwarzschild, Aniruddha Saha, Micah Goldblum, Jonas Geiping, and Tom Goldstein. NEFTune: Noisy Embeddings Improve Instruction Finetuning. In *ICLR*, 2024.

Xinyue Kang, Diwei Shi, and Li Chen. Model Whisper: Steering Vectors Unlock Large Language Models' Potential in Test-time. In *AAAI*, 2026.

Brian Lester, Rami Al-Rfou, and Noah Constant. The Power of Scale for Parameter-Efficient Prompt Tuning. In *EMNLP*, 2021.

Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. Solving Quantitative Reasoning Problems with Language Models. *arXiv:2206.14858*, 2022.

Xiang Lisa Li and Percy Liang. Prefix-Tuning: Optimizing Continuous Prompts for Generation. In *ACL-IJCNLP*, 2021.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's Verify Step by Step. In *ICLR*, 2024.

Xiao Liu, Yanan Zheng, Zhengxiao Du, Ming Ding, Yujie Qian, Zhilin Yang, and Jie Tang. GPT Understands, Too. *AI Open*, 5:208–215, 2024.

Aman Madaan and Amir Yazdanbakhsh. Text and Patterns: For Effective Chain of Thought, It Takes Two to Tango. *arXiv:2209.07686*, 2022.

Alexander Robey, Eric Wong, Hamed Hassani, and George J. Pappas. SmoothLLM: Defending Large Language Models Against Jailbreaking Attacks. *Transactions on Machine Learning Research*, 2025.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y.K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv:2402.03300*, 2024.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A Flexible and Efficient RLHF Framework. *arXiv:2409.19256*, 2024.

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*, 15:1929–1958, 2014.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.

Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xionghui Chen, Jianxin Yang, Zhenru Zhang, Yuqiong Liu, An Yang, Andrew Zhao, Yang Yue, Shiji Song, Bowen Yu, Gao Huang, and Junyang Lin. Beyond the 80/20 Rule: High-Entropy Minority Tokens Drive Effective Reinforcement Learning for LLM Reasoning. In *NeurIPS*, 2025.

Yige Xu, Xu Guo, Zhiwei Zeng, and Chunyan Miao. SoftCoT: Soft Chain-of-Thought for Efficient Reasoning with LLMs. In *ACL*, 2025.

An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, Keming Lu, Mingfeng Xue, Runji Lin, Tianyu Liu, Xingzhang Ren, and Zhenru Zhang. Qwen2.5-Math Technical Report: Toward Mathematical Expert Model via Self-Improvement. *arXiv:2409.12122*, 2024.

Wengao Ye, Yan Liang, and Lianlei Shan. Thinking on the Fly: Test-Time Reasoning Enhancement via Latent Thought Policy Optimization. In *ICLR*, 2026.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Juncai Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Ru Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Yonghui Wu, and Mingxuan Wang. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. In *NeurIPS*, 2025.

Weihao Zeng, Yuzhen Huang, Qian Liu, Wei Liu, Keqing He, Zejun Ma, and Junxian He. SimpleRL-Zoo: Investigating and Taming Zero Reinforcement Learning for Open Base Models in the Wild. *arXiv:2503.18892*, 2025.

Guibin Zhang, Muxin Fu, and Shuicheng Yan. MemGen: Weaving Generative Latent Memory for Self-Evolving Agents. In *ICLR*, 2026.

A Experimental Setup and Full Accuracy Breakdown

Table 6 provides the full per-position accuracy breakdown (Prefix, Infix, Suffix) corresponding to the best-across-positions results summarized in Table 1 in the main text. Table 7 lists the number of RSP tokens L used for each model–benchmark pair, selected from $\{10, 15, 20\}$. All experiments are conducted on a single NVIDIA A6000 40GB GPU with greedy decoding, a maximum generation length of 3,072 tokens for MATH-500 and GSM8K, and 4,096 tokens for AIME24. Batch size is fixed per model (16 for LLaMA-3.1-8B-Instruct and Qwen2.5-Math-7B variants, 32 for Qwen2.5-Math-1.5B variants).

Table 6: Accuracy (%) across model configurations, injection positions, and benchmarks. Values in parentheses indicate the difference from the baseline. **Bold** denotes the best result and $\underline{\text{underline}}$ denotes the second best for each model–benchmark pair.

Model	Mode	MATH-500	GSM8K	AIME24
Instruct Models				
LLaMA-3.1-8B-Instruct	Baseline	**52.20**	85.75	$\underline{6.67}$
	Prefix	45.00 (−7.2)	76.35 (−9.4)	3.33 (−3.3)
	Suffix	$\underline{49.40}$ (−2.8)	**86.73** (+1.0)	**10.00** (+3.3)
Qwen2.5-Math-7B-Instruct	Baseline	83.20	95.45	$\underline{13.33}$
	Prefix	81.40 (−1.8)	94.92 (−0.5)	$\underline{13.33}$ (+0.0)
	Suffix	**83.40** (+0.2)	**95.68** (+0.2)	**16.67** (+3.3)
Qwen2.5-Math-1.5B-Instruct	Baseline	73.00	$\underline{85.29}$	10.00
	Prefix	**74.80** (+1.8)	$\underline{85.29}$ (+0.0)	$\underline{13.33}$ (+3.3)
	Suffix	74.20 (+1.2)	84.69 (−0.6)	**20.00** (+10.0)
Base Models (Raw Text)				
Qwen2.5-Math-7B	Baseline	**72.20**	$\underline{84.15}$	6.67
	Prefix	$\underline{71.60}$ (−0.6)	**84.31** (+0.2)	**16.67** (+10.0)
	Suffix	55.60 (−16.6)	54.13 (−30.0)	$\underline{13.33}$ (+6.7)
Qwen2.5-Math-1.5B	Baseline	$\underline{61.40}$	**77.86**	**16.67**
	Prefix	**64.40** (+3.0)	$\underline{74.30}$ (−3.6)	$\underline{10.00}$ (−6.7)
	Suffix	54.60 (−6.8)	58.61 (−19.3)	3.33 (−13.3)
Base Models + ChatML (Format Mismatch)				
Qwen2.5-Math-7B	Baseline	52.20	58.30	**23.33**
	Prefix	59.20 (+7.0)	56.41 (−1.9)	$\underline{3.33}$ (−20.0)
	Suffix	**70.40** (+18.2)	**75.97** (+17.7)	**23.33** (+0.0)
Qwen2.5-Math-1.5B	Baseline	34.40	37.45	$\underline{13.33}$
	Prefix	**63.40** (+29.0)	**67.02** (+29.6)	**20.00** (+6.7)
	Suffix	$\underline{51.00}$ (+16.6)	$\underline{50.64}$ (+13.2)	$\underline{13.33}$ (+0.0)

Table 7: Number of RSP tokens (L) used for each model and benchmark.

Model	MATH-500	GSM8K	AIME24
LLaMA-3.1-8B-Instruct	15	20	15
Qwen2.5-Math-7B-Instruct	20	20	20
Qwen2.5-Math-1.5B-Instruct	20	10	20
Qwen2.5-Math-7B	10	10	20
Qwen2.5-Math-1.5B	10	10	20
Qwen2.5-Math-1.5B (ChatML)	20	20	10
Qwen2.5-Math-7B (ChatML)	20	20	20

A.1 RSP Injection Positions and Prompt Templates

Below we show the prompt structure for each injection position across all model types. Blue text indicates RSP embeddings, and purple text indicates special tokens.

A.1.1 Prefix

RSP embeddings are concatenated before the prompt embeddings. No text modification is applied.

ChatML (Qwen Instruct / Base + ChatML).

```
[Random Embeddings (L tokens)]
<|im_start|>system
Please reason step by step, and put your final answer within \boxed{}.<|im_end|>
<|im_start|>user
{question}<|im_end|>
<|im_start|>assistant
```

LLaMA Chat Template.

```
[Random Embeddings (L tokens)]
<|begin_of_text|>
<|start_header_id|>system<|end_header_id|>
Cutting Knowledge Date: December 2023
Today Date: 26 Jul 2024
Please reason step by step, and put your final answer within \boxed{}.<|eot_id|>
<|start_header_id|>user<|end_header_id|>
{question}<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
```

Raw Text (Qwen Base).

```
[Random Embeddings (L tokens)]
{question}
Please reason step by step, and put your final answer within \boxed{}.
```

A.1.2 Infix

A description text and special tokens are inserted into the prompt. The special token positions are then replaced with RSP embeddings at the embedding level.

ChatML (Qwen Instruct / Base + ChatML).

```
<|im_start|>system
Please reason step by step, and put your final answer within \boxed{}.<|im_end|>
<|im_start|>user
{question}
There are {L} special tokens that contain compressed latent reasoning information that
might be useful for your reasoning. If these tokens are useful for your case, you can
use them as reference. If these tokens are not useful for your case, you can ignore
them and focus back to solving the problem.
Here are the {L} special tokens: <special_token_1>...<special_token_L>
<|im_end|>
<|im_start|>assistant
```

LLaMA Chat Template.

```
<|begin_of_text|>
<|start_header_id|>system<|end_header_id|>
Cutting Knowledge Date: December 2023
Today Date: 26 Jul 2024
Please reason step by step, and put your final answer within \boxed{}.<|eot_id|>
<|start_header_id|>user<|end_header_id|>
{question}
There are {L} special tokens that contain compressed latent reasoning information that
might be useful for your reasoning. If these tokens are useful for your case, you can
use them as reference. If these tokens are not useful for your case, you can ignore
them and focus back to solving the problem.
Here are the {L} special tokens:
<reserved_special_token_0>...
<reserved_special_token_L>
```

```
<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
```

Raw Text (Qwen Base).

```
{question}
There are {L} special tokens that contain compressed latent reasoning information that
might be useful for your reasoning. If these tokens are useful for your case, you can
use them as reference. If these tokens are not useful for your case, you can ignore
them and focus back to solving the problem.
Here are the {L} special tokens: <special_token_1>...<special_token_L>
Please reason step by step, and put your final answer within \boxed{}.
```

A.1.3 Suffix

For chat-templated models, RSP embeddings are inserted immediately before the end-of-turn token. For raw text models, RSP embeddings are concatenated at the end of the prompt.

ChatML (Qwen Instruct / Base + ChatML).

```
<|im_start|>system
Please reason step by step, and put your final answer within \boxed{}.<|im_end|>
<|im_start|>user
{question}[Random Embeddings (L tokens)]<|im_end|>
<|im_start|>assistant
```

LLaMA Chat Template.

```
<|begin_of_text|>
<|start_header_id|>system<|end_header_id|>
Cutting Knowledge Date: December 2023
Today Date: 26 Jul 2024
Please reason step by step, and put your final answer within \boxed{}.<|eot_id|>
<|start_header_id|>user<|end_header_id|>
{question}[Random Embeddings (L tokens)]<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
```

Raw Text (Qwen Base).

```
{question}
Please reason step by step, and put your final answer within \boxed{}.[Random
Embeddings (L tokens)]
```

A.2 Answer Extraction and Grading

The final answer is extracted from the model output through a fallback chain. Each step is attempted in order; if extraction fails, the next step is tried.

Answer Extraction Fallback Chain

1. `"final answer is $...$. I hope"` pattern (Minerva format)
2. `\boxed{...}`: last non-empty box is used; empty `\boxed{}` are skipped
3. Text following `"the answer is"`
4. Text following `"final answer is"`
5. Last number in the output (regex fallback)

Extracted answers are normalized by removing `$`, `\left`, `\right`, and unit strings. Grading is performed via symbolic equivalence checking: exact string match, numerical comparison (`math.isclose`, `rel_tol=1e-4`), and SymPy symbolic simplification.

A.3 Reproduced Prior Work Hyperparameters

All prior methods are reproduced on Qwen2.5-Math-1.5B-Instruct and Qwen2.5-Math-7B-Instruct, evaluated with greedy decoding (temperature $= 0$) and our unified answer extraction pipeline.

TTSV. Prefix length 20, trained for 20 epochs with AdamW optimizer, batch size 2, gradient accumulation 8 steps, and linear learning rate schedule. Learning rate is 1e-3 for Qwen-1.5B and 1e-5 for Qwen-7B.

LTPO. Table 8 reports the per-benchmark hyperparameters.

Table 8: LTPO hyperparameters.

Parameter	GSM8K	MATH-500	AIME24
tokens	8	8	8
steps	20	20	20
top_k	10	10	10
sigma	20	20	5
sigma_decay	0.95	0.95	0.95
lr	1e-2	5e-3	5e-2

SoftCoT. Projection module trained for 10 epochs. Thought tokens: 32 during training, 4 during evaluation. Batch size 4 for GSM8K, 1 for MATH with gradient accumulation 4. The small (assistant) model is Qwen2.5-Math-1.5B-Instruct in both configurations, while the large model is Qwen2.5-Math-1.5B-Instruct (learning rate 2e-5) or Qwen2.5-Math-7B-Instruct (learning rate 5e-6). For MATH-500 and AIME24 evaluation, we use the projection module trained on GSM8K, as it yields higher accuracy than the one trained on MATH.

A.4 Full Entropy Curves

Figure 6 shows the full normalized (0–100%) generation trajectories for entropy, top-1 probability, and varentropy, and Table 9 reports the corresponding scalar statistics including overall means, first-5% means, and the average number of generated tokens per problem. All methods converge to similar levels after the initial generation stage, confirming that the distributional changes induced by RSP are localized to the early reasoning phase.

Table 9: Per-step statistics for Qwen2.5-Math-1.5B-Instruct on MATH-500 across the full generation and the first 5% of generation steps, together with the average number of generated tokens per problem. Top-1 Prob (overall) is reported as a weighted average over the first-10%/middle/last-10% segment means with weights $0.1/0.8/0.1$. Extension of Figure 3 (main text) and Figure 6.

Metric	Baseline	Prefix	Infix	Suffix	LTPO	SoftCoT	TTSV
Tokens (mean)	575.7	558.4	549.9	573.3	592.3	594.4	577.4
Entropy (first 5%)	0.1332	0.1366	0.1402	0.1941	0.1662	0.4218	0.1359
Entropy (overall)	0.0871	0.0881	0.0899	0.1049	0.0909	0.1059	0.0864
Top-1 Prob (first 5%)	0.9535	0.9523	0.9525	0.9373	0.9437	0.9156	0.9534
Top-1 Prob (overall)	0.9684	0.9681	0.9678	0.9647	0.9683	0.9651	0.9685
Varentropy (first 5%)	0.2181	0.2207	0.2463	0.4010	0.2953	2.2234	0.2174
Varentropy (overall)	0.1353	0.1375	0.1422	0.2038	0.1452	0.2404	0.1361

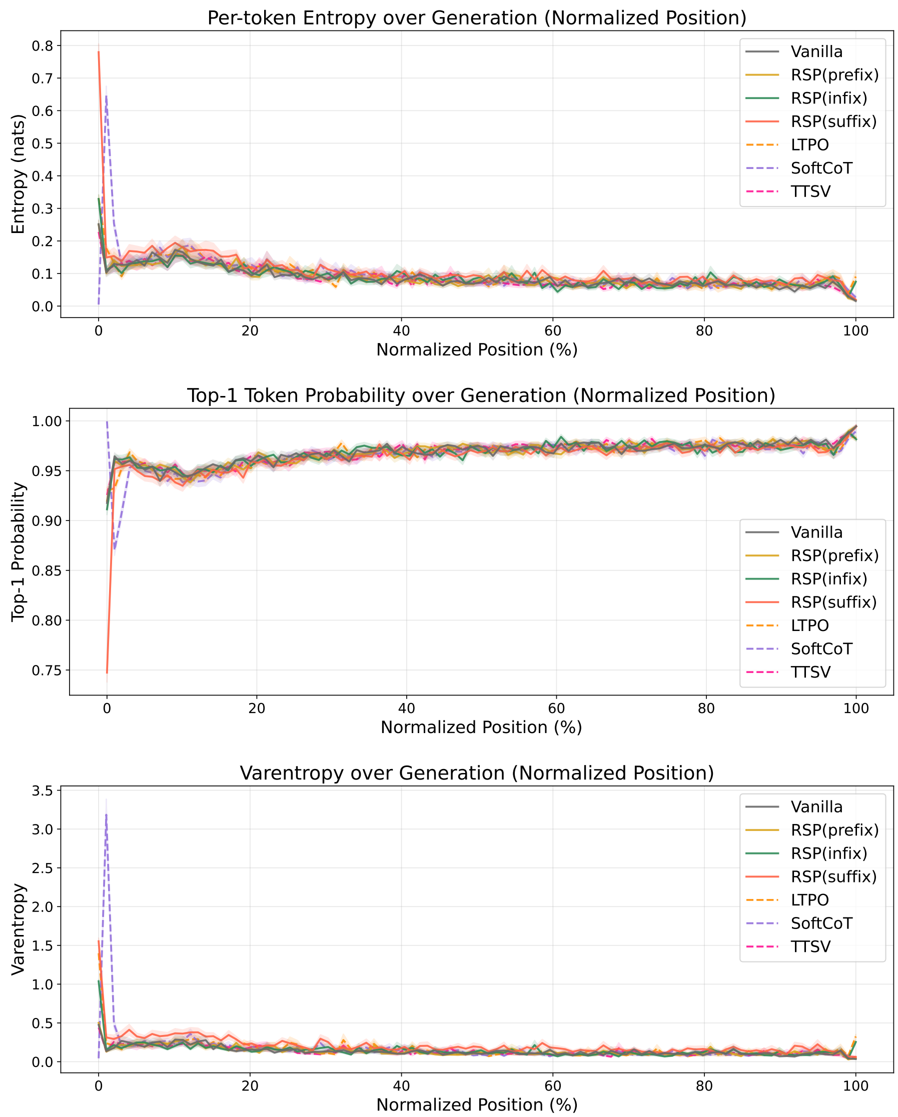


Figure 6: Mean entropy (top), top-1 probability (middle), and varentropy (bottom) over the full normalized generation trajectory (0–100%). Averaged over MATH-500, shading indicates ± 1 SEM. Solid lines: RSP variants and the baseline. Dashed lines: existing soft prompt methods. All methods converge after the initial stage.

B Token-Level Distribution Metrics

This appendix provides formal definitions for the metrics used in Section 4.4. Numerical results are reported in Table 3 of the main text. We extract first-token logits via a single forward pass (no autoregressive generation) and store the top-100 token ids and logit values per problem. Temperature conditions reuse the baseline logits scaled by $1/\tau$, requiring no separate inference. RSP draws 16 independent seeds per problem (MATH-500), and reported metrics are the mean across the 16 seeds. Pass@1 accuracy is computed over 16 samples per problem, with the baseline at τ=1.0 reaching $73.92 \pm 0.78\%$. Let P_{base} denote the baseline next-token distribution at τ=1.0 and P_{target} the candidate distribution being compared (e.g., baselines at τ=2.0, 3.0 or RSP at τ=1.0). All metrics are computed analytically from saved logits at the first generation step.

Spearman rank correlation. Spearman's ρ is the Pearson correlation coefficient between the token ranks of two distributions:

$$\rho \;=\; \text{Pearson}(\text{rank}(P_{\text{target}}),\; \text{rank}(P_{\text{base}}))\,. \tag{4}$$

Because temperature scaling $P^{(\tau)} \propto P^{1/\tau}$ is a strictly monotone transform, the rank order of tokens is preserved exactly, so $\rho = 1$ for any $\tau > 0$ as a mathematical identity. Any value $\rho < 1$ therefore indicates rank reorganization beyond what temperature scaling can produce.

Mass outside top-K. Let $\text{TopK}(P_{\text{base}})$ be the set of K tokens with the highest probability under P_{base}. The probability mass that the candidate distribution places *outside* this set is

$$\text{MassOutside}_K \;=\; 1 - \sum_{t \in \text{TopK}(P_{\text{base}})} P_{\text{target}}(t). \tag{5}$$

This directly measures support expansion: how much probability the candidate assigns to tokens that are not in the baseline's preferred set. Reported values use $K = 10$.

Jensen–Shannon divergence. The symmetrized, bounded version of Kullback–Leibler divergence:

$$\text{JS}(P, Q) \;=\; \tfrac{1}{2}\,\text{KL}(P \,\|\, M) + \tfrac{1}{2}\,\text{KL}(Q \,\|\, M), \qquad M = \tfrac{1}{2}(P + Q). \tag{6}$$

We use base-2 logarithms so that JS $\in [0, 1]$. Tokens absent from the stored top-100 are assigned a sentinel logit (-10^9) before softmax, which is negligible for our setting.

C Prompt-Law Properties Used by the Theory

This appendix isolates the only properties of the RSP law used in the main text: centeredness, direction-agnostic covariance under a variance budget, and positive probability on every nonempty open set. We avoid stronger semantic claims because Section 3.4 needs only these geometric and measure-theoretic facts.

C.1 Centered Perturbations Do Not Impose Systematic First-Order Drift

Let Δ_{ij} be the baseline logit gap between tokens i and j, and write the first-order surrogate as

$$\Delta_{ij}(\bar{h}) = \Delta_{ij} + b_{ij}^\top \bar{h}.$$

Proposition 3 (No systematic first-order drift)**.** *If* $\mathbb{E}[\bar{h}] = 0$*, then*

$$\mathbb{E}\big[\Delta_{ij}(\bar{h})\big] = \Delta_{ij}$$

for every token pair (i, j)*. If a deterministic prompt* h_0 *is reused across runs, then*

$$\Delta_{ij}(h_0) - \Delta_{ij} = b_{ij}^\top h_0$$

is a fixed directional bias.

Proof. The centered case is immediate from linearity of expectation:

$$\mathbb{E}\big[\Delta_{ij}(\bar{h})\big] = \Delta_{ij} + b_{ij}^\top \mathbb{E}[\bar{h}] = \Delta_{ij}.$$

The deterministic case follows by substitution. □

This proposition rules out a run-averaged first-order bias. It does not say that individual prompt draws leave the logits unchanged.

C.2 Proof of Theorem 2

For a unit vector b, the first-order perturbation energy available along b is $\mathrm{Var}_{h\sim D}(b^\top h) = b^\top \Sigma_D b$. The worst-case directional energy is therefore the minimum Rayleigh quotient of Σ_D.

Proof. For any symmetric covariance matrix Σ_D,

$$\min_{\|b\|=1} b^\top \Sigma_D b = \lambda_{\min}(\Sigma_D).$$

Let the eigenvalues of Σ_D be $\lambda_1, \ldots, \lambda_d$. Then

$$\lambda_{\min}(\Sigma_D) \le \frac{1}{d}\sum_{m=1}^{d} \lambda_m = \frac{\mathrm{tr}(\Sigma_D)}{d} \le \frac{\rho^2}{d}.$$

Equality holds if and only if all eigenvalues are equal, i.e., $\Sigma_D = (\rho^2/d)\mathbf{I}_d$. □

The proof uses only covariance. We choose a Gaussian law for RSP because it adds the open-set reachability property proved next.

C.3 Open-Set Reachability of Isotropic Gaussian Prompts

Proposition 4 (Open-set reachability)**.** *Let* $\bar{h} \sim \mathcal{N}(0, \sigma^2\mathbf{I}_d)$ *with* $\sigma > 0$*. Then every nonempty open set* $U \subseteq \mathbb{R}^d$ *satisfies*

$$\Pr(\bar{h} \in U) > 0.$$

Proof. The Gaussian density

$$(2\pi\sigma^2)^{-d/2} \exp\left(-\frac{\|\bar{h}\|^2}{2\sigma^2}\right)$$

is strictly positive at every point of $\mathbb{R}^d$. Every nonempty open set contains a ball of positive Lebesgue measure, and integrating a strictly positive density over that ball gives positive probability. □

This is the only support property used in Proposition 1.

D Proofs for the Transformer-Level Mechanism

This appendix follows the same order as the main theory section: exploration, annealing, and performance gain. The prompt-law facts used before these proofs are collected in Appendix C.

D.1 Exploration: attention decomposition and perturbation size

Proposition 5 (Attention decomposition)**.** *For one attention head at query position i in layer ℓ, let $\alpha_{ij}^{(\ell)}$ be the attention weights over $n \geq 1$ unmasked real tokens and $L \geq 1$ random tokens, with finite attention logits. Define*

$$w_{r,i}^{(\ell)} := \sum_{j=1}^{L} \alpha_{i,n+j}^{(\ell)}$$

and, for $j \in [n]$,

$$\tilde{\alpha}_{ij}^{(\ell)} := \frac{\alpha_{ij}^{(\ell)}}{1 - w_{r,i}^{(\ell)}}.$$

Then

$$\mathbf{o}_i^{(\ell)} = (1 - w_{r,i}^{(\ell)})\, \tilde{\mathbf{o}}_i^{(\ell)} + \boldsymbol{\eta}_i^{(\ell)},$$

where

$$\tilde{\mathbf{o}}_i^{(\ell)} := \sum_{j=1}^{n} \tilde{\alpha}_{ij}^{(\ell)} \mathbf{v}_j^{(\ell)} \qquad \textit{and} \qquad \boldsymbol{\eta}_i^{(\ell)} := \sum_{j=1}^{L} \alpha_{i,n+j}^{(\ell)} \mathbf{v}_{r_j}^{(\ell)}.$$

Proof. The head output is

$$\mathbf{o}_i^{(\ell)} = \sum_{j=1}^{n} \alpha_{ij}^{(\ell)} \mathbf{v}_j^{(\ell)} + \sum_{j=1}^{L} \alpha_{i,n+j}^{(\ell)} \mathbf{v}_{r_j}^{(\ell)}.$$

By the softmax positivity and the existence of at least one unmasked real token, $\sum_{j=1}^{n} \alpha_{ij}^{(\ell)} = 1 - w_{r,i}^{(\ell)} > 0$, so

$$\sum_{j=1}^{n} \alpha_{ij}^{(\ell)} \mathbf{v}_j^{(\ell)} = (1 - w_{r,i}^{(\ell)}) \sum_{j=1}^{n} \frac{\alpha_{ij}^{(\ell)}}{1 - w_{r,i}^{(\ell)}} \mathbf{v}_j^{(\ell)} = (1 - w_{r,i}^{(\ell)})\, \tilde{\mathbf{o}}_i^{(\ell)}.$$

Substituting this identity gives the decomposition. □

Corollary 1 (Magnitude scales with random-token attention)**.** *For every realization of the random values,*

$$\|\boldsymbol{\eta}_i^{(\ell)}\| \leq w_{r,i}^{(\ell)} \max_{j \in [L]} \|\mathbf{v}_{r_j}^{(\ell)}\|.$$

Proof. By the triangle inequality,

$$\|\boldsymbol{\eta}_i^{(\ell)}\| = \left\| \sum_{j=1}^{L} \alpha_{i,n+j}^{(\ell)} \mathbf{v}_{r_j}^{(\ell)} \right\| \leq \sum_{j=1}^{L} \alpha_{i,n+j}^{(\ell)} \|\mathbf{v}_{r_j}^{(\ell)}\| \leq w_{r,i}^{(\ell)} \max_{j \in [L]} \|\mathbf{v}_{r_j}^{(\ell)}\|.$$

□

Corollary 1 is the only quantitative fact needed in the main text. No independence assumption between attention weights and random keys is required. Once $w_{r,i}^{(\ell)}$ is small, the random contribution must be small as well.

D.2 Corollaries of Theorem 1 on cache growth

The fixed-gap decay bound yields the corollaries below.

Corollary 2 (Sequence-wise annealing of the fixed-gap envelope)**.** *For fixed L and $\Delta_i^{(\ell)}$, the upper bound*

$$f(n) = \frac{L}{L + n\exp(\Delta_i^{(\ell)}/\sqrt{d})}$$

is strictly decreasing in n and tends to 0 as $n \to \infty$. Cache growth alone therefore narrows the largest RSP attention mass compatible with that gap.

During autoregressive decoding $\Delta_i^{(\ell)}$ also moves because queries, keys, and hidden states co-evolve. The accurate reading is not "the realized mass decreases at every step" but "the envelope compatible with a fixed gap shrinks as the cache grows."

Proof. Differentiate $f(n)$ with respect to n:

$$f'(n) = -\frac{L\exp(\Delta_i^{(\ell)}/\sqrt{d})}{\Big(L + n\exp(\Delta_i^{(\ell)}/\sqrt{d})\Big)^2} < 0.$$

Thus $f(n)$ is strictly decreasing, and its denominator diverges linearly in n while its numerator is fixed, so $f(n) \to 0$. □

Corollary 3 (Vanishing random contribution under annealing)**.** *If $n \to \infty$ while L and $\Delta_i^{(\ell)}$ remain fixed, $w_{r,i}^{(\ell)} \to 0$, and for every realization of the random values*

$$\|\boldsymbol{\eta}_i^{(\ell)}\| \le w_{r,i}^{(\ell)} \max_{j\in[L]} \|\mathbf{v}_{r_j}^{(\ell)}\| \to 0.$$

Proof. $w_{r,i}^{(\ell)} \to 0$ by Corollary 2. Apply Corollary 1 to bound $\|\boldsymbol{\eta}_i^{(\ell)}\|$. The maximum norm is finite for any realization of finitely many sampled values. □

D.3 Performance gain: from local admission to Pass@N

The main text §3.4 uses a first-order surrogate to state two claims: a baseline-excluded token can enter the local top-K set, and independent reruns can turn such local openings into Pass@N gains. The proofs below use only one support condition on the centered prompt law D:

$$\text{(S1)} \qquad D(U) > 0 \quad \text{for every nonempty open set } U \subseteq \mathbb{R}^d.$$

Proposition 4 shows that the isotropic Gaussian law of §3.4 satisfies (S1).

D.3.1 Autoregressive Formulation

In autoregressive decoding, the joint distribution over a sequence $y_{1:T}$ factorizes as

$$\Pr(y_{1:T} \mid x) = \prod_{t=1}^{T} p(y_t \mid x, y_{<t}),$$

where each step is governed by a prefix-conditioned distribution. Any perturbation introduced by RSP affects generation through a sequence of local changes to $p(y_t \mid x, y_{<t})$, rather than a single global distribution.

D.3.2 Local Linearization of Logits under RSP

Throughout this subsection, $\bar{h} \in \mathbb{R}^d$ denotes *one* centered prompt vector from the RSP definition (Eq. (1) in §3.1), treated as a per-token surrogate. The actual implementation uses a length-L prompt $\mathbf{H} = (h_1, \ldots, h_L)$ with L independent draws; the arguments below isolate how one local perturbation can shift the logits. We model the perturbed logits as a function $z_i(\bar{h})$, assuming local smoothness in a neighborhood of $\bar{h} = 0$:

$$z_i(\bar{h}) = z_i(0) + \nabla_h z_i(0)^\top \bar{h} + r_i(\bar{h}),$$

where $r_i(\bar{h}) = O(\|\bar{h}\|^2)$. Defining $a_i := \nabla_h z_i(0)$, we obtain the local affine surrogate

$$z_i^{\mathrm{lin}}(\bar{h}) := z_i + a_i^\top \bar{h}.$$

This approximation is used as a first-order surrogate for branch opening, not as an exact global model of the transformer logits.

D.3.3 Proof of Proposition 1

Let

$$\mathcal{G}_{i,K} \ := \ \big\{\bar{h} \in \mathbb{R}^d : \#\{j \neq i : z_j^{\mathrm{lin}}(\bar{h}) > z_i^{\mathrm{lin}}(\bar{h})\} \leq K - 1\big\}.$$

Proof. Assume $\mathcal{G}_{i,K}$ contains a nonempty open set U. By (S1), $D(U) > 0$. Since $U \subseteq \mathcal{G}_{i,K}$,

$$\Pr_{\bar{h}}(\bar{h} \in \mathcal{G}_{i,K}) \ \geq \ D(U) \ > \ 0.$$

This is exactly the event that token i enters the surrogate top-K set.

A useful sufficient condition is the existence of some $\bar{h}_0$ at which token i strictly outranks all but at most $K - 1$ other tokens in the affine surrogate. By continuity, a neighborhood of $\bar{h}_0$ is then contained in $\mathcal{G}_{i,K}$. □

D.3.4 Proof of Proposition 2

This proposition does not derive $p_{\min}(x) > 0$ from the mechanism. It only converts a task-side lower bound and independence into Pass@N growth.

Proof. Let A_n be the event that the n-th independent RSP run reaches a correct trajectory on problem x, and write $p_n(x) := \Pr(A_n) \geq p_{\min}(x)$ as in (M1). Under (M2),

$$\Pr\Big(\bigcap_{n=1}^{N} A_n^c\Big) = \prod_{n=1}^{N}(1 - p_n(x)) \leq \prod_{n=1}^{N}(1 - p_{\min}(x)) = (1 - p_{\min}(x))^N.$$

Taking complements gives

$$\Pr\big(\text{Pass@}N \text{ on } x\big) = \Pr\Big(\bigcup_{n=1}^{N} A_n\Big) \geq 1 - (1 - p_{\min}(x))^N.$$

The bound $1 - x \leq e^{-x}$ for $x \in [0, 1]$ then yields

$$1 - (1 - p_{\min}(x))^N \ \geq \ 1 - e^{-N p_{\min}(x)}.$$

□

E Attention Mass Analysis

This appendix formalizes the per-token attention mass reported in Figure 2 and the exclusion criteria applied to the reasoning span and the layer axis. For each of the 500 MATH-500 problems, a fresh RSP $\mathbf{H} \in \mathbb{R}^{L \times d}$ with $L = 10$ is sampled independently, so the reported heatmaps average over both problem variability and RSP-vector variability.

E.1 Per-Token Attention Mass

For each problem, we measure attention on the concatenated sequence [prompt; $\mathbf{H}$; generation], where the generation is the trajectory produced by the same model under matching RSP injection. Let $\alpha_{i,j}^{(\ell,h)}$ denote the post-softmax attention weight at layer ℓ, head h, from query position i to key position j, satisfying $\sum_j \alpha_{i,j}^{(\ell,h)} = 1$ under the causal mask. Let Q and R denote the index sets of question and RSP tokens with sizes $|Q|$ and $|R| = L$, and let H be the number of heads. The per-token attention mass that query i directs to each group at layer ℓ is

$$\mathrm{PTM}_Q[\ell, i] = \frac{1}{|Q|H} \sum_{h=1}^{H} \sum_{j \in Q} \alpha_{i,j}^{(\ell,h)}, \qquad \mathrm{PTM}_R[\ell, i] = \frac{1}{|R|H} \sum_{h=1}^{H} \sum_{j \in R} \alpha_{i,j}^{(\ell,h)}. \tag{7}$$

Normalization by $|Q|$ and $|R|$ controls for group size, so both quantities express how much attention a single question (resp. RSP) token receives on average under the same softmax denominator. Question tokens thereby serve as a size-matched baseline that absorbs sequence-length effects common to both groups.

E.2 Heatmap Aggregation

We partition the layer indices and the reasoning position indices each into five equal-size bins $\{B_L^{(b)}\}_{b=1}^5$ and $\{B_P^{(b)}\}_{b=1}^5$. For sample s and target group $\bullet \in \{Q, R\}$, the value in cell (b_L, b_P) is

$$M_\bullet^{(s)}[b_L, b_P] = \frac{1}{|B_L^{(b_L)}| \cdot |B_P^{(b_P)}|} \sum_{\ell \in B_L^{(b_L)}} \sum_{i \in B_P^{(b_P)}} \mathrm{PTM}_\bullet[\ell, i]. \tag{8}$$

Figure 2 reports the sample average of Eq. (8) over the 500 valid samples.

E.3 Excluding the `\boxed{...}` Span

The reasoning position range terminates strictly before the final `\boxed{...}` span. Chain-of-thought outputs decompose into a *text* (content) component and a *pattern* (template) component that contribute independently to downstream performance [Madaan and Yazdanbakhsh, 2022]. The `\boxed{...}` wrapper is a canonical pattern: a short, stereotyped span whose role is to complete the answer format rather than to extend reasoning. Including it would therefore inject a template-driven attention signature unrelated to reasoning dynamics.

E.4 Excluding the Final Two Layers

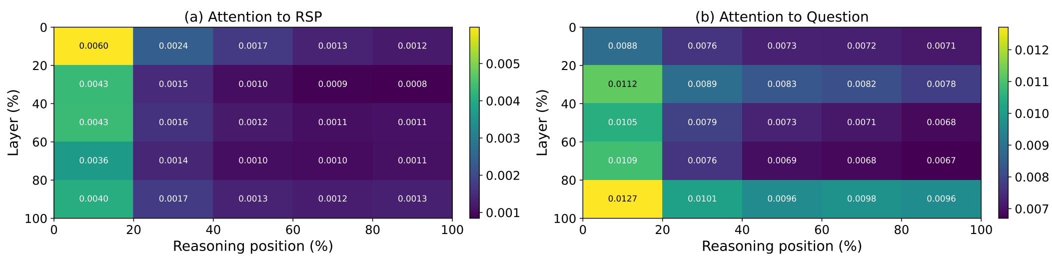
The layer axis further excludes the last two transformer layers. This choice is motivated by a standard late-layer effect rather than by any model-specific tuning decision.

Output-projection specialization. Late-layer hidden states lie in an approximately linear relationship with vocabulary logits and therefore function as a progressive linear readout rather than as representation-transforming blocks [Belrose et al., 2023]. Consistently, late-layer feed-forward modules have been characterized as output-distribution shaping mechanisms [Geva et al., 2021]. Attention in these layers therefore reflects output formation rather than reasoning computation.

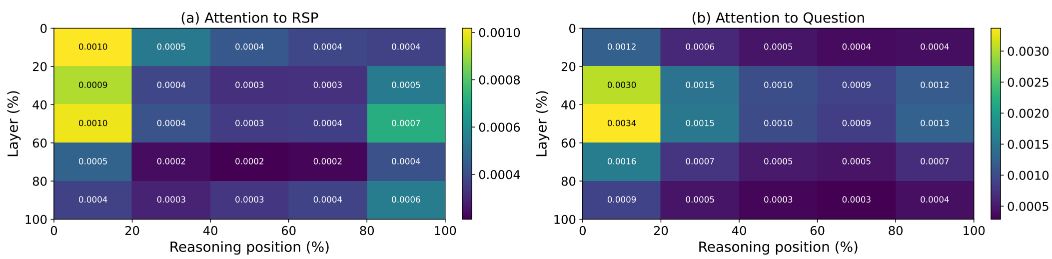
Excluding the final two layers keeps the heatmap focused on reasoning-phase dynamics instead of readout-phase dynamics. This exclusion is not load-bearing for the main claim: the sequence-direction attenuation emphasized in the main text is also visible without it, and Theorem 1 concerns sequence-direction attenuation rather than layer-wise monotonicity.

E.5 Additional Suffix Heatmaps

Figure 7 reports the same per-token RSP attention analysis under suffix injection for Qwen2.5-Math-1.5B-Instruct and LLaMA-3.1-8B-Instruct. For the late-layer reasons discussed above, the pattern does not generalize uniformly across every cell; nevertheless, the overall trend is consistent across both models: question-token attention varies broadly across layers, whereas RSP attention shows the layer-wise and sequence-wise decay predicted by Theorem 1.



(a) Qwen2.5-Math-1.5B-Instruct, suffix



(b) LLaMA-3.1-8B-Instruct, suffix

Figure 7: Per-token RSP attention mass under suffix injection for the remaining two models (500 MATH-500 samples each). Axes and preprocessing match Figure 2: x-axis is reasoning position binned into five quantiles, y-axis is layer depth binned into five quantiles (top = shallow), color encodes the 500-sample mean per-token attention assigned to RSP tokens, and tokens inside `\boxed{}` spans and the last two layers are excluded.

F DAPO Implementation

This appendix complements Section 5 with the DAPO loss modifications relative to GRPO, the DAPO + RSP implementation, and full per-step results.

F.1 From GRPO to DAPO

The vanilla GRPO objective [Shao et al., 2024] for a group of G rollouts $\{o_i\}_{i=1}^G$ with rewards $\{R_i\}_{i=1}^G$ is

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\Big(\min\big(r_{i,t}\hat{A}_{i,t},\, \text{clip}(r_{i,t}, 1-\varepsilon, 1+\varepsilon)\hat{A}_{i,t}\big) - \beta\, D_{\text{KL}}(\pi_\theta \| \pi_{\text{ref}})\Big)\right], \tag{9}$$

with importance ratio $r_{i,t} = \pi_\theta(o_{i,t} \mid q, o_{i,<t})/\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})$, group-relative advantage $\hat{A}_{i,t} = (R_i - \text{mean}(\{R_i\}))/\text{std}(\{R_i\})$, clipping range ε, and KL penalty $\beta\, D_{\text{KL}}$ against the reference π_{ref}.

We build on the SimpleRL-Zoo [Zeng et al., 2025] training pipeline and implement DAPO on top of VeRL [Sheng et al., 2024]. DAPO modifies Eq. (9) in four ways: (i) token-level loss aggregation $1/\sum_i |o_i|$ instead of $1/|o_i|$, (ii) asymmetric clipping with $(\varepsilon_{\text{low}}, \varepsilon_{\text{high}}) = (0.2, 0.28)$, (iii) dual clipping safeguard $\max(L_{\text{clipped}}, c\hat{A})$ with $c = 10$, and (iv) KL terms removed. The resulting DAPO objective is

$$\mathcal{J}_{\text{DAPO}}(\theta) \;=\; \mathbb{E}\left[\frac{1}{\sum_i |o_i|}\sum_{i,t}\min\Big(r_{i,t}\hat{A}_{i,t},\, \text{clip}\big(r_{i,t},\, 1-\varepsilon_{\text{low}},\, 1+\varepsilon_{\text{high}}\big)\hat{A}_{i,t}\Big)\right]. \tag{10}$$

F.2 DAPO + RSP Implementation

For DAPO + RSP, we additionally inject a group-specific RSP $\mathbf{H}_g \in \mathbb{R}^{L\times d}$ ($L = 20$) into each rollout. $\mathbf{H}_g$ is generated deterministically from a per-step group seed and injected via a forward hook; it is not a trainable parameter, so the learning signal is the policy adapting to arbitrary $\mathbf{H}_g$ rather than a learned prompt. Including $\mathbf{H}_g$ in the log-probabilities at both rollout and update time keeps the update on-policy with respect to the RSP-conditioned distribution (avoiding off-policy mismatch). With $G = 8$ and $n = 8$, each group contains a single response, so each rollout effectively sees a different $\mathbf{H}_g$.

F.3 Data, Batch, and Hyperparameters

Table 10: DAPO / DAPO + RSP training setup on Qwen2.5-Math-7B.

Field	**Value**
Train data	MATH Level 3–5 subset (`simplelr_math_35`, ~8,500 prompts)
Prompt template	`qwen-boxed`
Train batch size	1024
PPO mini-batch size	256 (4 PPO updates per rollout)
Rollouts per prompt G	8
Max prompt / response length	2,028 / 2,048
Rollout sampling	temperature 1.0, top-p 1.0, top-k 50
$(\varepsilon_{\text{low}}, \varepsilon_{\text{high}})$	$(0.2,\ 0.28)$
Dual clip c	10.0
Loss aggregation	token-mean ($1/\sum_i |o_i|$)
KL terms	disabled
Entropy coefficient	0
Optimizer	AdamW, learning rate 5×10^{-7} (constant), no warmup
Total training	~10 epochs, ≥ 80 optimization steps
Save / eval frequency	every 10 steps
RSP (DAPO + RSP only)	suffix injection, $L = 20$, freshly drawn per rollout
Hardware	4× NVIDIA B200 GPUs
Wall-clock training time	$\sim$12 hours per run

F.4 Evaluation Protocol

Each saved checkpoint is converted to HuggingFace format and evaluated using the SimpleRL-Zoo `eval_math_nodes.sh` pipeline, which selects sampling parameters per benchmark to reduce variance on the smaller competition sets:

Table 11: Per-benchmark evaluation protocol used by SimpleRL-Zoo `sh/eval.sh`.

Benchmark	# Problems	Temperature	n_{sampling}	Metric
AIME 2024	30	1.0	32	Avg@32
AMC 2023	40	1.0	32	Avg@32
MATH-500	500	0.0	1	accuracy (mean@1)
GSM8K	1,319	0.0	1	accuracy
OlympiadBench	675	0.0	1	accuracy
Minerva Math	272	0.0	1	accuracy
GaoKao 2023-EN	385	0.0	1	accuracy

The maximum number of generated tokens is 16,000 across all benchmarks. The reported average is the unweighted mean over the five benchmarks (GSM8K, MATH-500, College Math, Minerva Math, AIME 2024).

F.5 Per-Step Average Accuracy

Table 12: Five-benchmark average accuracy (%) at every checkpoint, computed over GSM8K, MATH-500, College Math, Minerva Math, and AIME 2024. **Bold** marks each method's peak.

Step	DAPO	DAPO + RSP
0	30.06	30.06
10	46.40	46.22
20	49.02	49.54
30	49.58	50.72
40	50.62	52.06
50	50.88	52.40
60	51.84	52.96
70	52.52	53.86
80	**53.20**	54.04
90	52.52	**54.36**
100	50.54	53.64

G Broader Impacts

This work provides empirical and theoretical analysis of how random embedding injection affects the reasoning behavior of large language models. The contributions are primarily foundational, with potential downstream implications for reinforcement learning-based post-training of reasoning models such as GRPO.

Positive impacts. RSP-induced trajectory diversity could improve the sample efficiency of RL-based LLM post-training by providing richer learning signals within group-relative advantage estimation. This has the potential to reduce compute costs associated with alignment and reasoning-oriented fine-tuning, making such methods more accessible to research groups with limited resources.

Potential concerns. Methods that expand the effective output support of LLMs may increase the reachability of low-probability tokens, which includes both desirable alternative reasoning paths and potentially unsafe or off-distribution outputs. Practitioners applying RSP in production systems should combine it with existing safety filtering and alignment mechanisms rather than treating it as a standalone diversity source.

Limitations on impact scope. Since this work focuses on analysis rather than deploying a new model or dataset, the direct societal impact is limited. The broader implications described above are contingent on future work integrating RSP into applied training pipelines.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and §1 state four scoped claims, each tied to a section: (i) RSP's contribution at each attention layer is captured by a single scalar with a fixed-gap upper-envelope cache-growth bound (Theorem 1, §3.3); (ii) under a local affine surrogate, isotropic re-sampling reaches top-K entry regions unattainable by rank-preserving temperature sampling (§3.4); (iii) RSP raises early-token entropy and stabilizes later, with Pass@N gains under independent re-sampling (§4); and (iv) the diversity transfers to RL training when combined with DAPO (§5). We do not claim a theorem that directly explains the format-mismatch recovery (Table 1); the wrong-attractor hypothesis there is explicitly informal.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Limitations are noted in several places rather than in a dedicated section: (a) the multi-path Pass@N argument requires the task-side assumption $p_{\min}(x) > 0$, flagged in §3.4 and §6; (b) the $+18$–29 pp recovery on the format-mismatch rows of Table 1 is empirically observed but not formally derived from any theorem in §3, with the wrong-attractor account stated as a hypothesis to be quantified in future work (§4.2); (c) Table 1 reports the best-position accuracy across $\{$Prefix, Infix, Suffix$\}$, and per-position variance is large with several cases below baseline (Appendix A), so position is treated as a hyperparameter rather than something we justify principally; (d) DAPO + RSP results (§5) use a single training seed under a reduced DAPO configuration that omits Dynamic Sampling and overlong reward shaping; (e) main accuracy comparisons are single-seed without bootstrap or significance tests (item 7). Future work in §6 addresses training-time comparisons, domain expansion, and principled position selection.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [Yes]

   Justification: Theorem 1 (cache-growth attenuation) carries an inline proof in §3.3. Proposition 2 is stated in §3.4 with its proof in Appendix D; the supporting top-K entry and Pass@N ensemble arguments around it are presented as informal discussion in §3.4 with formal write-ups in the appendix. Each statement explicitly lists its assumptions: finite logits and $n, L \geq 1$ for Theorem 1; finite second-moment budget for Proposition 2; and the locally affine surrogate plus $p_{\min}(x) > 0$ task-side condition together with mutual independence of the N runs for the Pass@N argument.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: §4.1 and Appendix A specify benchmarks (MATH-500, GSM8K, AIME 2024 in the main table; further sets used for the DAPO study), models (LLaMA-3.1-8B-Instruct and Qwen2.5-Math 1.5B/7B variants), the RSP injection grid (Prefix/Infix/Suffix; $L \in \{10, 15, 20\}$), greedy decoding, and the answer extraction pipeline. Appendix F provides the full DAPO + RSP training recipe (batch of $1{,}024$ prompts, $G = 8$ rollouts per prompt at $\tau = 1.0$, four PPO mini-batch updates of 256 prompts each, 100 training steps, suffix RSP $L = 20$ during rollouts only).

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: All datasets and base models used in the paper are publicly available and cited (MATH-500, GSM8K, AIME 2024, AMC 2023, OlympiadBench, Minerva Math, GaoKao 2023-EN, College Math; Qwen2.5-Math 1.5B/7B and LLaMA-3.1-8B-Instruct). RSP itself is described to a level of detail sufficient for re-implementation (Eq. (1) and §4.1). However, our RSP injection harness and DAPO + RSP training pipeline are not yet released; we will release an anonymized implementation alongside the camera-ready submission.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: §4.1 and Appendix A list all benchmarks, models, RSP injection settings (positions, lengths, per-row L choice in Appendix Table 7), maximum generation length per benchmark, and decoding strategy. Appendix F provides the full DAPO + RSP training settings, including optimizer, batch composition, mini-batch updates, asymmetric clipping range $(\varepsilon_{\text{low}}, \varepsilon_{\text{high}}) = (0.2, 0.28)$, dual clipping safeguard, and RSP injection protocol during rollouts.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [No]

   Justification: Pass@N experiments use $N = 16$ samples per problem (§4.4), Table 3 reports mean$\pm$standard deviation over 16 RSP seeds, and AIME 2024 is scored by Avg@32 to reduce variance. However, Table 1 (Table 1) reports single-run accuracies without bootstrap confidence intervals or paired tests, and the DAPO + RSP training in §5 uses a single training seed. Multi-seed evaluation, particularly on small competition sets (AIME 2024, AMC 2023) where variance is highest, is left for future work.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Inference experiments (Tables 1, 2; Figure 3) run on a single NVIDIA A6000 40GB GPU with greedy decoding (Appendix A). DAPO + RSP training (§5) runs on $4\times$ NVIDIA B200 GPUs for approximately 12 hours per run (Appendix F, hardware row of the configuration table).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work uses publicly available math reasoning benchmarks and pretrained language models, involves no human subjects or personal data, and does not introduce a release that bypasses standard model-safety considerations. We have reviewed the NeurIPS Code of Ethics and identify no conflicts.

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

Justification: Appendix G discusses both directions: positive impact via more sample-efficient RL post-training of reasoning LLMs, and a potential downside that broadening the effective output support also increases reachability of undesirable trajectories, which we argue should keep RSP coupled with existing safety filtering rather than replacing it.

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper does not release new pretrained models, generative systems, or scraped datasets. RSP is a generation-time perturbation applied to existing pretrained models and introduces no separate artifact requiring safeguards.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: All datasets (MATH-500 [Lightman et al., 2024], GSM8K [Cobbe et al., 2021], AIME 2024, AMC 2023, OlympiadBench [He et al., 2024], Minerva Math [Lewkowycz et al., 2022], GaoKao 2023-EN, College Math) and pretrained models (Qwen2.5-Math [Yang et al., 2024], LLaMA-3.1 [Grattafiori et al., 2024]) are publicly available and cited at first use. The training pipeline builds on SimpleRL-Zoo [Zeng et al., 2025] and VeRL [Sheng et al., 2024], also cited. Each asset is used under its original publicly stated terms.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [N/A]

    Justification: The paper does not release new datasets or pretrained models. The implementation code planned for camera-ready release (item 5) will be accompanied by configuration files and a README; no derived dataset or model checkpoint is part of the submission.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The paper involves no crowdsourcing and no research with human subjects. All evaluations use static, publicly available math reasoning benchmarks.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: As noted in item 14, the paper involves no human subjects, so IRB review is not applicable.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: Pretrained LLMs (Qwen2.5-Math, LLaMA-3.1) appear in the paper as the experimental subject of analysis, not as a non-standard component of the methods themselves. The methods (RSP definition, attention decomposition, DAPO + RSP training) do not rely on auxiliary LLMs in any non-standard or originality-affecting role.